

Nexus SDLC — Complete Installation & Setup Guide

This guide covers **everything** you need to install Nexus globally, configure it in any workspace, set up the knowledge base, and install the VS Code extension.

Table of Contents

- 1. [Prerequisites](#)
- 2. [Global Installation \(Terminal Agent\)](#)
- 3. [Setting Up a New Workspace](#)
- 4. [Skills / Knowledge Base Configuration](#)
- 5. [VS Code Extension Installation](#)
- 6. [Advanced Configuration](#)
- 7. [Troubleshooting](#)

1. Prerequisites

Before installing Nexus, ensure you have:

Tool	Version	Check Command
Python	3.10+	<code>python --version</code>
pip	Latest	<code>pip --version</code>
Git	2.x+	<code>git --version</code>
Node.js	18+ (only for VS Code ext dev)	<code>node --version</code>

Azure OpenAI Access (Required for AI features)

You need an Azure OpenAI resource with:

- An API endpoint URL
- An API key
- A deployed model (default: `gpt-4.1`)

Note: Without Azure OpenAI, Nexus runs in **mock mode** — all commands work but AI responses are simulated.

2. Global Installation (Terminal Agent)

Step 1: Clone the Repository

```
git clone https://github.com/chinmayshete/sdlc-terminal.git
```

```
cd sdlc-terminal
```

Step 2: Install Python Dependencies

```
pip install -r requirements.txt
```

Step 3: Install Nexus Globally

```
pip install -e .
```

This installs the **nexus** command globally. The **-e** flag makes it editable, so updates to the source code take effect immediately.

Step 4: Verify Installation

```
# From any directory
nexus --version
nexus terminal      # Starts the interactive REPL
nexus serve         # Starts the API server for VS Code extension
```

What Gets Installed

```
nexus terminal  → Interactive Rich REPL with 5 modes
nexus serve     → FastAPI bridge server (for VS Code extension)
nexus tickets   → List Jira tickets (CLI mode)
nexus plan <id> → Generate plan (CLI mode)
nexus execute <id> → Execute ticket (CLI mode)
nexus status    → Show workspace status
nexus ai        → Check AI health
nexus scan      → Run security scan
```

3. Setting Up a New Workspace

When you want to use Nexus in a **new project**, you need to create some configuration files in that project's root directory.

Step 1: Navigate to Your Project

```
cd /path/to/your-project
```

Step 2: Create the `.env` File

Create a `.env` file in the project root with your credentials:

```
# _____  
# .env - Nexus SDLC Configuration  
# _____  
  
# — Azure OpenAI (REQUIRED for AI features) —  
AZURE_OPENAI_ENDPOINT=https://your-resource.openai.azure.com  
AZURE_OPENAI_API_KEY=your-api-key-here  
AZURE_OPENAI_DEPLOYMENT=gpt-4.1  
AZURE_OPENAI_API_VERSION=2024-12-01-preview  
  
# Set to "true" to run without Azure OpenAI (simulated responses)  
SDLC_USE MOCK=false  
  
# — Jira Integration (OPTIONAL) —  
JIRA_HOST=your-org.atlassian.net  
JIRA_EMAIL=your-email@company.com  
JIRA_API_TOKEN=your-jira-api-token  
JIRA_PROJECT_KEY=PROJ  
  
# — Confluence Integration (OPTIONAL) —  
CONFLUENCE_SPACE_KEY=PROJ  
CONFLUENCE_BASE_URL=https://your-org.atlassian.net/wiki/spaces/PROJ/pages/12345  
  
# — Application Environment —  
APP_ENV=dev  
  
# — Vault Integration (OPTIONAL) —  
VAULT_ENABLED=false  
VAULT_ADDR=http://127.0.0.1:8200  
VAULT_TOKEN=  
VAULT_SECRET_PATH=secret/data/sdlc  
  
# — SSL (set to true if behind corporate proxy) —  
SDLC_SKIP_SSL_VERIFY=false
```

⚠ IMPORTANT: Never commit `.env` to Git. Add it to `.gitignore`.

Step 3: Create the `.sdlc/` State Directory

Nexus automatically creates this directory, but you can pre-create it:

```
mkdir .sdlc
```

This directory stores:

- `state.json` — Ticket workflow statuses (TODO → PLANNED → IN_DEVELOPMENT → COMPLETED)
- `scans/` — Security scan history and cached results

Step 4: Create the `tickets/` Directory (Optional)

```
mkdir tickets
```

When you run `nexus plan <ticket-id>`, the generated plans are saved here as markdown files (e.g., `tickets/AUTH-101_plan.md`).

Step 5: Create the `config/` Directory (Optional — for Multi-Environment Config)

```
mkdir -p config/environments
```

Create `config/base.json`:

```
{
  "azure": {
    "endpoint": "",
    "deployment": "gpt-4.1",
    "api_key": "",
    "api_version": "2024-12-01-preview"
  },
  "features": {
    "use_mock": false,
    "enable_code_scanning": true,
    "enable_cicd_integration": true
  },
  "logging": {
    "level": "info",
    "format": "json"
  },
  "database": {
    "host": "localhost",
    "port": "5432",
    "name": "nexus",
    "ssl": "false",
    "username": "",
    "password": ""
  },
  "services": {
    "jenkins_url": "",
    "sonarqube_url": "",
    "vault_addr": "http://127.0.0.1:8200"
  }
}
```

Create environment-specific overrides (e.g., `config/environments/dev.json`):

```
{
  "features": {
    "use_mock": false
  },
  "logging": {
    "level": "debug"
  }
}
```

Minimum Required Workspace Structure

```
your-project/
├── .env                ← REQUIRED (Azure OpenAI + Jira credentials)
├── .gitignore          ← Should include .env, .sdlc/
├── .sdlc/              ← Auto-created (workflow state)
│   └── state.json
├── tickets/            ← Auto-created (generated plans)
├── Skills/             ← OPTIONAL (knowledge base)
└── config/             ← OPTIONAL (multi-env config)
    ├── base.json
    └── environments/
        ├── dev.json
        ├── staging.json
        └── prod.json
```

Step 6: Start Nexus

```
# Terminal mode
nexus terminal

# Or just check health
nexus ai
```

4. Skills / Knowledge Base Configuration

The `Skills/` folder is Nexus's **knowledge base**. Any `.md`, `.txt`, `.yaml`, or `.json` files placed here are automatically loaded into the AI agent's context, so it reasons in line with your team's standards.

Step 1: Create the Skills Directory

```
mkdir -p Skills/engineering
mkdir -p Skills/security
```

```
mkdir -p Skills/devops
mkdir -p Skills/sdlc
```

Step 2: Add Knowledge Base Documents

The recommended structure:

```
Skills/
├── README.md                ← Overview of your KB
├── coding_standards_kb.md   ← Top-level coding standards
├── software_development_practices_kb.md ← Top-level dev practices
├── engineering/
│   ├── coding_standards.md  ← Code quality & architecture
│   ├── api_design.md        ← Language-specific conventions
│   ├── testing_strategy.md  ← REST API design patterns
│   └── security_practices.md ← Unit/integration test strategy
│                               ← Secure coding guidelines
├── security/
│   └── sast_rules_and_guardrails.md ← Security governance
│                                       ← SAST scanning rules & policies
├── devops/
│   ├── cicd_pipeline.md     ← CI/CD & infrastructure
│   └── deployment_checklist.md ← Pipeline stages & best practices
│                                       ← Pre-deploy verification steps
└── sdlc/
    ├── definition_of_done.md ← Process & workflow
    ├── sprint_workflow.md    ← What "done" means for a ticket
    ├── story_definition.md   ← Sprint ceremony guidelines
    └── ticket_planning.md    ← How to write user stories
                                ← Planning methodology
```

Step 3: Example Knowledge Base Files

Skills/engineering/coding_standards.md

```
# Coding Standards

## Python
- Use type hints on all function signatures
- Follow PEP 8 naming conventions
- Maximum function length: 50 lines
- All public functions must have docstrings
- Use `async/await` for I/O operations

## TypeScript
- Use strict mode (`strict: true` in tsconfig)
- Prefer interfaces over type aliases for object shapes
- Use `const` by default, `let` only when reassignment is needed
```

```
## General
- No hardcoded secrets or API keys
- All error messages must be user-friendly
- Log at appropriate levels (DEBUG, INFO, WARNING, ERROR)
```

Skills/security/sast_rules_and_guardrails.md

```
# SAST Rules & Guardrails

## Critical Rules (Block Deployment)
- No hardcoded credentials (API keys, passwords, tokens)
- No SQL injection vulnerabilities
- No XSS in user-facing outputs
- No insecure deserialization

## Warning Rules (Require Review)
- Missing input validation on API endpoints
- Overly permissive CORS configuration
- Missing rate limiting on public endpoints
```

Skills/sdlc/definition_of_done.md

```
# Definition of Done

A ticket is "Done" when ALL of the following are true:
1. Code is implemented and compiles without errors
2. Unit tests written with >80% coverage on new code
3. Security scan passes with zero ERROR findings
4. Code reviewed by at least 1 team member
5. Documentation updated if API changes were made
6. Deployed to staging and smoke-tested
```

How the Agent Uses Skills

Command	Skills Usage
plan <ticket>	Loads skills matching ticket keywords (e.g., "auth" → security_practices.md)
execute <ticket>	Loads skills before code generation to ensure output matches team standards
Freeform chat	ALL skills are loaded so you can ask "what's our testing strategy?"

Customizing for Your Team

The default skills contain generic best practices. **Edit them** to reflect:

- Your team's specific coding conventions
- Your CI/CD pipeline structure

- Your security policies and compliance requirements
- Your definition of done and sprint processes

5. VS Code Extension Installation

Option A: Install Pre-built VSIX

The VSIX file is at [vscode-extension/nexus-sdlc-0.1.0.vsix](#).

Step 1: Install the Extension

1. Open VS Code
2. Press `Ctrl+Shift+P` → "Extensions: Install from VSIX..."
3. Navigate to: `sdlc-terminal/vscode-extension/nexus-sdlc-0.1.0.vsix`
4. Click "Install"
5. Reload VS Code when prompted

Step 2: Start the API Server

The VS Code extension communicates with a Python API server. You need to start it:

```
# In your project directory (where .env is)
cd /path/to/your-project
nexus serve
```

This starts the API server on <http://localhost:9500>.

Auto-start: The extension is configured to auto-start the server. If Python is on your PATH, it will attempt `python -m src.cli.program serve` in the workspace folder.

Step 3: Use the Extension

-  **Activity bar icon** — Click the lightning bolt in the sidebar to open Nexus Chat
- **Ctrl+Shift+N** — Quick open the Nexus chat
- **Status bar** — Shows server connection status and current mode
- **Ctrl+Shift+P** — Type "Nexus:" to see all available commands

Option B: Build from Source

```
cd sdlc-terminal/vscode-extension

# Install dependencies
npm install

# Compile
```

```
npm run compile

# Package
npx @vscode/vsce package --allow-missing-repository

# Install the generated .vsix
code --install-extension nexus-sdlc-0.1.0.vsix
```

Extension Settings

Configure in VS Code Settings (**Ctrl+,**):

Setting	Default	Description
<code>nexus.serverUrl</code>	<code>http://127.0.0.1:9500</code>	URL of the Nexus API server
<code>nexus.autoStartServer</code>	<code>true</code>	Auto-start the server on extension activation
<code>nexus.pythonPath</code>	<code>python</code>	Path to Python (e.g., <code>python3</code> or full path)
<code>nexus.serverPort</code>	<code>9500</code>	Port for the API server

VS Code Commands

Command	Keybinding	Description
Nexus: Open Chat	<code>Ctrl+Shift+N</code>	Open/focus the sidebar chat
Nexus: Switch Mode	—	QuickPick to switch between 5 modes
Nexus: Run Security Scan	—	Full workspace security scan
Nexus: Scan Current File	Right-click menu	Scan the active file
Nexus: Show Tickets	—	Browse & action on Jira tickets
Nexus: Git Status	—	Quick git status
Nexus: Health Check	—	AI service health check
Nexus: Start API Server	—	Start the Python server
Nexus: Stop API Server	—	Stop the Python server
Nexus: Restart API Server	—	Restart the Python server

6. Advanced Configuration

Jira API Token

To get a Jira API token:

1. Go to <https://id.atlassian.com/manage-profile/security/api-tokens>
2. Click "Create API token"

3. Copy the token to `JIRA_API_TOKEN` in `.env`

Confluence Integration

Nexus can pull project requirements from Confluence:

1. Set `CONFLUENCE_SPACE_KEY` to your space key
2. Set `CONFLUENCE_BASE_URL` to the page URL containing your requirements
3. The agent will automatically fetch Confluence docs as context when planning tickets

Running Behind a Corporate Proxy

If you're behind a corporate proxy with SSL inspection:

```
SDLC_SKIP_SSL_VERIFY=true
```

Using Multiple Workspaces

Each workspace is independent. Navigate to any project and Nexus uses that directory's:

- `.env` for credentials
- `Skills/` for knowledge base
- `.sdlc/` for state
- `tickets/` for plans

```
# Workspace A (React frontend)
cd ~/projects/frontend
nexus terminal

# Workspace B (Python API)
cd ~/projects/api
nexus terminal
```

7. Troubleshooting

"Azure config missing" / Mock mode

Cause: `.env` file not found or Azure credentials not set. **Fix:** Ensure `.env` exists in the current working directory with valid `AZURE_OPENAI_ENDPOINT` and `AZURE_OPENAI_API_KEY`.

"nexus: command not found"

Cause: Global install not on PATH. **Fix:** Run `pip install -e .` from the `sdlc-terminal` root, or add the Python scripts directory to your PATH.

VS Code extension shows "Offline"

Cause: API server not running. **Fix:** Run `nexus serve` in your project directory, or use the command palette: "Nexus: Start API Server".

Jira tickets not loading

Cause: Jira credentials missing or invalid. **Fix:** Verify `JIRA_HOST`, `JIRA_EMAIL`, `JIRA_API_TOKEN`, and `JIRA_PROJECT_KEY` in `.env`.

Skills not loading

Cause: Folder name case mismatch. **Fix:** Nexus looks for the `skills/` folder (case-insensitive on Windows, case-sensitive on Linux). The path is configured in `src/config/paths.py` as `root_dir / "skills"`.

Security scan takes too long

Cause: Large workspace with many files. **Fix:** Add directories to the skip list in `.gitignore`, or use `scan file <path>` for targeted scanning.

Quick Reference Card

NEXUS QUICK REFERENCE

INSTALL: `pip install -e .`

TERMINAL: `nexus terminal`

API SERVER: `nexus serve [--port 9500]`

VS CODE: Install `nexus-sdlc-0.1.0.vsix`

WORKSPACE SETUP:

1. Create `.env` (Azure + Jira creds)

2. Create `Skills/` (knowledge base)

3. Run: `nexus terminal`

MODES:

`nexus > ____` Main hub / AI chat

`git > ____` Version control

`security > ____` SAST & governance

`devops > ____` CI/CD & infra

`agile > ____` Jira & sprints

KEY FILES:

`.env` Credentials

`Skills/` Knowledge base

`.sdlc/` Workflow state

`tickets/` Generated plans

`config/` Multi-env config